UNIVERSIDAD DE GUAYAQUIL

FACULTAD DE CIENCIAS, MATEMATICAS Y FISICAS

CARRERA DE SOFTWARE

MATERIA:

APLICACIONES DISTRIBUIDAS

TEMA DE ACTIVIDAD:

RPC, RMI Y SOCKETS

DOCENTE:

CALDERON GAVILANES MARLON ADRIAN

ESTUDIANTE:

TENESACA PINCAY CINTHIA MARIA

CICLO ESCOLAR:

2025 – 2026 CICLO 2

INDICE

1. RPC (REMOTE PROCEDURE CALL) 1

1.1. Definición 1

1.2. Como funciona 1

1.3. Tipos de RPC 1

1.4. Ventajas y Desventajas 2

2. RMI (REMOTE METHOD INVOCATION) 2

2.1. Definición 2

2.2. Soluciones 3

2.3. Funcionamiento 3

2.4. Pasos para la implementación 3

2.5. Ventajas y Desventajas 4

3. Sockets 4

3.1. Definición 4

3.2. Tipos 4

3.3. Ventajas y Desventajas 5

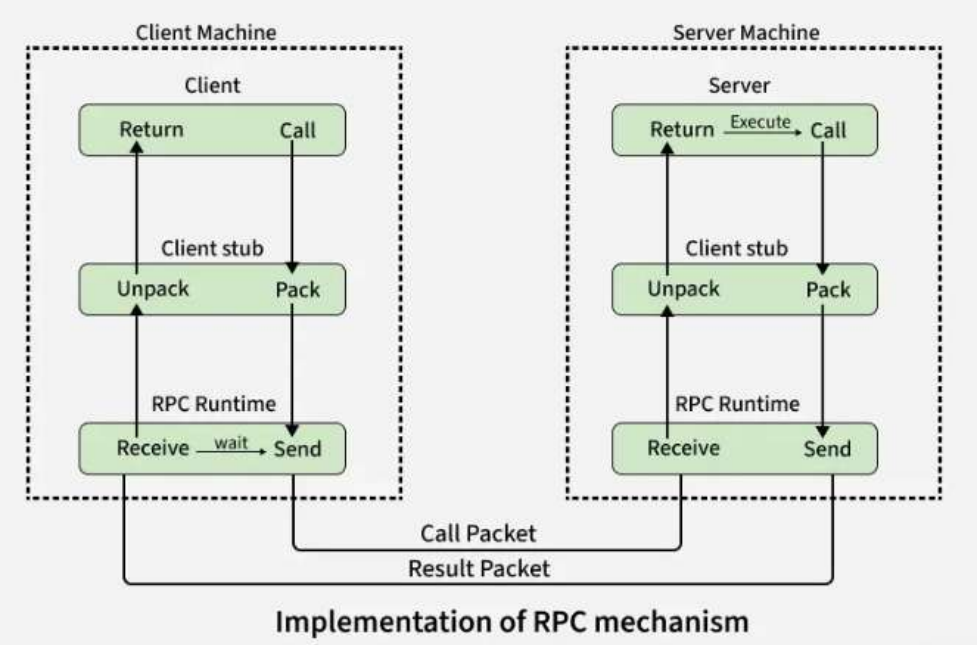
Bibliografía 6

RPC, RMI Y SOCKETS

1. RPC (REMOTE PROCEDURE CALL)

1.1.Definición

Abstrae la comunicación a nivel de invocación de procedimientos. Es adecuada para programación estructurada basada en librerías



1.2.Como funciona

- **El cliente** llama a un procedimiento local (stub) como si fuera normal.
- **Marshalling** : el stub empaqueta (agrupa) todos los parámetros de entrada en un mensaje.
- **Enviar al servidor** : el mensaje se envía a través de la red al servidor.
- **Stub del servidor:** el stub del servidor descomprime el mensaje y llama al procedimiento del servidor real.
- **Ejecución y retorno:** el servidor ejecuta el procedimiento y devuelve el resultado al stub.
- **De vuelta al cliente:** el stub del servidor envía el resultado de vuelta y el stub del cliente lo descomprime.

1.3.Tipos de RPC

- **RPC de devolución de llamada** : Tanto el cliente como el servidor pueden actuar el uno como el otro. El servidor puede devolver la llamada al cliente, lo

cual es útil en aplicaciones interactivas. Gestiona los interbloqueos y admite la comunicación punto a punto

- **RPC de difusión** : La solicitud del cliente se difunde a todos los servidores. Se utilizan puertos de difusión especiales. Resulta útil cuando varios servidores pueden gestionar la solicitud.
- **RPC por lotes** : Agrupa múltiples solicitudes de cliente y las envía juntas al servidor, lo que reduce la sobrecarga de la red. Ideal para aplicaciones con llamadas poco frecuentes.

1.4.Ventajas y Desventajas

VENTAJAS	DESVENTAJAS
• ***Comunicación sencilla:*** RPC permite a los clientes comunicarse con los servidores mediante llamadas a procedimientos normales en lenguajes de programación de alto nivel . Esto facilita el trabajo de los programadores • ***Complejidad oculta:*** RPC oculta los detalles del envío de mensajes entre computadoras. Esto significa que los programadores no tienen que preocuparse por la comunicación de red subyacente. • ***Flexibilidad:*** RPC puede utilizarse tanto en entornos locales como distribuidos, lo que lo hace versátil para diferentes tipos de aplicaciones.	• ***Paso limitado de parámetros:*** RPC solo puede pasar parámetros por valor. No puede pasar punteros, lo que limita lo que se puede enviar entre computadoras • ***Más lento que las llamadas locales:*** las llamadas a procedimientos remotos toman más tiempo que las llamadas a procedimientos locales porque involucran comunicación de red. • ***Vulnerabilidad a fallos:*** RPC depende de conexiones de red, otras máquinas y procesos independientes. Esto aumenta su probabilidad de fallos en comparación con las llamadas a procedimientos locales.

2. RMI (REMOTE METHOD INVOCATION)

2.1.Definición

La Invocación Remota de Métodos (RMI) es una API que permite que un objeto invoque un método en un objeto existente en otro espacio de direcciones, ya sea en

la misma máquina o en una máquina remota. Mediante RMI, un objeto que se ejecuta en una JVM presente en un equipo (lado del cliente) puede invocar métodos en un objeto presente en otra JVM (lado del servidor). RMI crea un objeto de servidor remoto público que permite la comunicación entre el cliente y el servidor mediante llamadas a métodos simples en el objeto de servidor.

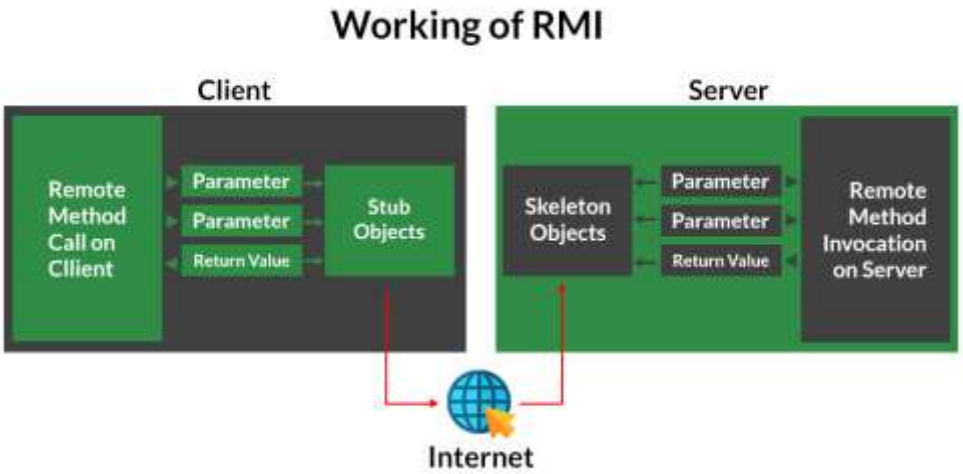
2.2.Soluciones

Solución Java para la comunicación de objetos Java distribuidos.

- Solución simple (Fácil de uso)
- Solución natural (Se comporta como se espera que haga)

2.3.Funcionamiento

La comunicación entre el cliente y el servidor se gestiona mediante dos objetos intermedios: el objeto Stub (en el lado del cliente) y el objeto Skeleton (en el lado del servidor), como también se puede representar en los siguientes medios:



2.4.Pasos para la implementación

1. Definición de una interfaz remota
2. Implementando la interfaz remota
3. Creación de objetos Stub y Skeleton desde la clase de implementación usando rmic (compilador RMI)
4. Iniciar rmiregistry
5. Crear y ejecutar el programa de aplicación del servidor
6. Crear y ejecutar el programa de aplicación del cliente

2.5.Ventajas y Desventajas

VENTAJAS	DESVENTAJAS
• Permite distribuir una aplicación de forma muy transparente, es decir, sin que el programador tenga que modificar apenas el código. • Las invocaciones remotas son más eficientes que las peticiones vía http que se usan con los CGIs o los Servlets.	• El paso de parámetros por valor implica tiempo para hacer la serialización, enviar los objetos serializados a través de la red y luego volver a recomponer los objetos en el destino

3. Sockets

3.1.Definición

Facilitan la generación dinámica de canales de comunicación. Es actualmente la base de la comunicación. Pero al ser de muy bajo nivel de abstracción, no son adecuados a nivel de aplicación

3.2.Tipos

- ***Sockets de flujo (stream sockets):*** Proporcionan una conexión bidireccional confiable y orientada a la secuencia. Son ideales para aplicaciones que requieren una transmisión continua de datos, como la transmisión de audio o vídeo en tiempo real. El protocolo TCP (Transmission Control Protocol), por ejemplo, suele asociarse con esta categoría.
- ***Sockets de datagrama (datagram sockets):*** Ofrecen un servicio no orientado a la conexión y envían datos en paquetes independientes. Son adecuados para aplicaciones donde la entrega inmediata de datos es crucial, pero la pérdida ocasional no es crítica. Un ejemplo de protocolo que suele utilizarse con este tipo es UDP (User Datagram Protocol).

3.3.Ventajas y Desventajas

VENTAJAS	DESVENTAJAS
• Permite intercambiar flujo de datos entre un cliente – servidor de manera fiable y ordenada. • Se beneficia de la funcionalidad API.	• Es dependiente de la respuesta del servidor. • En el tipo de sockets no orientado a la conexión, el protocolo UPD no garantiza que los datos lleguen en el mismo orden en que se enviaron.

Bibliografía

Drake, J. M. (s.f.). *PROGRAMACION CONCURRENTE Y DISTRIBUIDA - VII.1: RMI: Remote Method Invocation*. Obtenido de UNICAN: https://www.ctr.unican.es/asignaturas/procodis_3_II/Doc/Procodis_7_01.pdf

Llamada a procedimiento remoto (RPC) en el sistema operativo. (25 de octubre de 2025). Obtenido de geeksforgeeks: https://www.geeksforgeeks.org/operating-systems/remote-procedure-call-rpc-in-operating-system/

Mallón, X. (s.f.). *¿Qué es un socket, cómo funciona y qué tipos de sockets hay?* Obtenido de keppcoding: https://keepcoding.io/blog/que-es-un-socket/

Remote Method Invocation in Java. (11 de Enero de 2023). Obtenido de geeksforgeeks: https://www.geeksforgeeks.org/java/remote-method-invocation-in-java/